

Apostila de Expressões e Termos Utilizados na Programação em Python

✨ Introdução

Esta apostila foi criada com o objetivo de auxiliar iniciantes no aprendizado da linguagem de programação **Python**, apresentando os principais termos, expressões e comandos utilizados na linguagem, explicando para que servem e quando devem ser utilizados.

📘 Primeiros Passos com Python

Se você está iniciando agora, o ideal é começar a praticar sem precisar instalar nada no computador. Para isso, recomendamos o uso de **IDEs online** (ambientes de programação que funcionam direto do navegador).

🔢 IDEs Online para Começar

- https://www.onlinegdb.com
- https://replit.com
- https://trinket.io
- https://www.programiz.com/python-programming/online-compiler
- https://pythonanywhere.com
- https://my.raspberrypi.org/dashboard

Vantagens:

- Não precisa instalar nada.
- Funciona direto no navegador.
- Ideal para testes e primeiros aprendizados.

📚 Quando se familiarizar com os primeiros códigos...

Aí sim é hora de instalar uma **IDE no seu computador**, para ter mais recursos, velocidade e praticidade para projetos maiores.

📂 **IDEs para instalar**

- **Thonny**: Leve, simples, ideal para iniciantes (https://thonny.org)
- **VS Code**: Muito poderosa, usada por iniciantes e profissionais (https://code.visualstudio.com)
- **PyCharm**: IDE completa da JetBrains (https://www.jetbrains.com/pycharm/)

🧠 O que é Sintaxe em Programação?

Sintaxe é o conjunto de regras que define como escrever comandos corretamente em uma linguagem de programação, como o Python.

> É como a gramática da linguagem.

🗣️ Exemplo com Linguagem Natural (Português)

- Correto: "Eu gosto de estudar Python."
- Incorreto: "Estudo eu gosto Python de." → A ordem das palavras não faz sentido.

💻 Exemplo com Python

✅ Sintaxe correta:

python

CopiarEditar

```
print("Olá, mundo!")
```

❌ Sintaxe errada:

python

CopiarEditar

```
print "Olá, mundo!"  # Erro! Faltam os parênteses.
```

🛑 O que acontece se você errar a sintaxe?

O Python mostra um erro de sintaxe (SyntaxError), e o programa não roda até você corrigir.

🔎 Termos e Expressões

1. Variáveis

- **O que são:** Espaços na memória usados para armazenar valores.
- **Sintaxe:** `nome_da_variavel = valor`
- **Quando usar:** Quando precisar guardar um valor para uso posterior.
- **Exemplo:** `idade = 12`

2. Tipos de Dados

- **int:** Números inteiros. Ex: `5`, `-3`
- **float:** Números com ponto decimal. Ex: `3.14`, `-0.5`
- **str:** Texto (strings). Ex: `'Olá'`, `"Python"`
- **bool:** Valores booleanos: `True` ou `False`
- **Quando usar:** Para representar diferentes tipos de informação.

3. Operadores Aritméticos

- `+` (adição), `-` (subtração), `*` (multiplicação), `/` (divisão), `//` (divisão inteira), `%` (módulo), `**` (exponenciação)
- **Quando usar:** Para realizar cálculos matemáticos.

4. Funções

- **O que são:** Blocos de código reutilizáveis que executam uma tarefa.
- **Sintaxe:**

```
def nome_da_funcao():
    # instruções
```

- **Quando usar:** Quando quiser reutilizar um conjunto de instruções.

5. Condicionais (if, elif, else)

- **O que fazem:** Permitem executar códigos diferentes com base em condições.
- **Sintaxe:**

```
if condição:
    # instruções
elif outra condição:
    # instruções
else:
    # instruções
```

- **Quando usar:** Quando precisar tomar decisões no código.

6. Laços de Repetição (for, while)

- **for:** Repete um bloco de código um número determinado de vezes.
- **while:** Repete enquanto uma condição for verdadeira.
- **Sintaxe:**

```
for i in range(5):
    print(i)

while condicao:
    # instruções
```

- **Quando usar:** Para repetir tarefas de forma automática.

7. Listas

- **O que são:** Estruturas que guardam vários valores.
- **Sintaxe:** `nomelista = [1, 2, 3]`
- **Quando usar:** Para armazenar coleções de dados.

8. Dicionários

- **O que são:** Estrutura de dados que armazena pares chave:valor.

- **Sintaxe:** `aluno = {"nome": "Ana", "idade": 10}`
- **Quando usar:** Quando quiser associar uma chave a um valor.

9. Input e Output

- `input()` para receber dados do usuário.
- `print()` para exibir mensagens na tela.
- **Exemplo:**

```
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

10. Comentários

- **O que são:** Linhas que explicam o código, mas não são executadas.
- **Sintaxe:** `# isso é um comentário`
- **Quando usar:** Para tornar o código mais compreensível.

🤖 **Palavras-chave (Keywords) do Python**

- **and, or, not**: Operadores lógicos usados para combinar ou negar condições.
- **as**: Usado para dar um "apelido" a módulos importados.
- **assert**: Verifica se uma condição é verdadeira. Caso contrário, lança um erro.
- **break**: Interrompe um laço de repetição.
- **class**: Declara uma classe (Program.ção Orientada a Objetos).
- **continue**: Pula para a próxima iteração do laço.
- **def**: Define uma função.
- **del**: Remove variáveis ou itens.
- **elif, else, if**: Estruturas condicionais.
- **except, try, finally**: Tratamento de exceções (erros).
- **False, True**: Valores booleanos.
- **for, while**: Laços de repetição.
- **from, import**: Importa módulos ou partes deles.

- **global, nonlocal**: Declarar variáveis que estão fora do escopo local.
- **in, is**: `in` verifica se um valor está contido; `is` compara identidade de objetos.
- **lambda**: Cria funções anônimas.
- **None**: Representa a ausência de valor.
- **pass**: Instrução vazia (placeholder).
- **raise**: Lança uma exceção (erro).
- **return**: Retorna um valor de uma função.
- **with**: Gerência contexto, como abertura de arquivos.
- **yield**: Cria geradores, retornando valores sem encerrar a função.

Nota: Esses são os principais termos que você vai encontrar ao começar a programar em Python. Com a prática, você vai entender cada um deles de forma natural. Use esta apostila como um guia rápido sempre que tiver dúvidas.

Cezar Augusto Trevisan

professorcezar50@gmail.com